Taller Teórico: Fundamentos y arquitectura de datos en JavaScript

Jimena Sofía Valiente Blandón

Institución Universitaria De Colombia

Ingeniería de Software – 4to semestre

Ingeniería Web II (JavaScript – Web Services)

Profesor: Juan Carlos Arbeláez Rodríguez

26 de febrero de 2026

ÍNDICE

Contenido

Introducción 3
Desarrollo del taller 4
1. Alcance y Mutabilidad (Scope & Mutability) 4
2. Tipado Dinámico vs. Estático 5
3. Tipos de Datos Primitivos 5
4. Ausencia de Valor: null vs. undefined 6
5. Coerción de Tipos (Type Coercion) 7
6. El operador typeof y su anomalía histórica 7
7. Elevación (Hoisting) y la Zona Muerta Temporal (TDZ) 8
8. Reglas de Identificadores (Naming Conventions) 9
9. La "Inmutabilidad" relativa de const 9
10. La paradoja de NaN 10
Conclusiones 11
Bibliografía 12

Introducción

Aprender JavaScript hoy en día va mucho más allá que solo memorizarse los comandos, se trata de entender cómo funciona realmente el lenguaje que mueve toda la web moderna. Aunque al inicio parece un lenguaje fácil, JavaScript tiene ciertas reglas y comportamientos que han ido evolucionando con los años para que sea más ordenado y profesional.

Este trabajo busca resolver preguntas y explorar conceptos fundamentales que todo desarrollador web debe dominar para hacer programas de calidad. A lo largo de este taller, veremos desde cómo el navegador gestiona la memoria y el alcance de las variables, hasta esos detalles curiosos como el Hoisting o la coerción de tipos, que muchas veces podrían causar errores si no se comprenden bien.

Desarrollo del taller

1. Alcance y Mutabilidad (Scope & Mutability) **Explique la diferencia fundamental en cuanto a "alcance" (scope) y mutabilidad entre las palabras reservadas var, let y const. ¿Por qué el estándar moderno (ES6+) desaconseja el uso de var?**

Rta// la palabra reservada “var” tiene un alcance de función o alcance global, es decir, si se declara una variable usando “var” dentro de un bloque (if o for), esta sigue existiendo fuera de este bloque.

En la mutabilidad de la palabra “var” esta puede ser reasignada y modificada.

Para la palabra reservada “let”, su alcance es de bloque, es decir, esta es limitada al bloque en que se encuentra y solo existe dentro de { }.

Y su mutabilidad, esta puede ser reasignada pero no redeclarable en el mismo bloque.

Para “const”, su alcance es igual que let, es decir, que solo tiene block scope.

Y la palabra “const” a diferencia de las demás es inmutable, es decir que no se puede reasignar o redeclarar.

El estándar moderno desaconseja el uso de “var” por su alcance, ya que esta no va a respetar el alcance de bloques como los bucles o condicionales, lo que puede producir problemas en bloques grandes de código. Además, también el problema común de “var” es el hoisting o elevación, ya que JavaScript eleva la declaración de var al comienzo, lo que permite que se usen antes de declararse formalmente.

2. Tipado Dinámico vs. Estático **JavaScript es un lenguaje de tipado dinámico (débilmente tipado). Defina qué significa este concepto en la arquitectura del lenguaje y compárelo brevemente con el comportamiento de un lenguaje de tipado estático (como C# o Java) al momento de declarar y reasignar variables.**

Rta// El tipado dinámico en JavaScript significa que no necesitamos declarar el tipo, ya que este identifica el tipo de variable que estamos declarando y la asigna a pesar de que no sea de forma explícita.

A diferencia de los lenguajes de tipado estático como C# o Java, es obligatorio declarar el tipo de dato y este no puede modificarse después, así que JavaScript da la flexibilidad de reasignar cualquier valor a una variable previamente declarada, en este se ve como el tipo está ligado al valor, es decir, el tipo de la variable puede cambiar a medida que cambian el tipo de valores que se le asignan.

3. Tipos de Datos Primitivos **El estándar ECMAScript define un conjunto específico de tipos de datos que no son objetos y no tienen métodos propios. Enumere los siete (7) tipos de datos primitivos que existen actualmente en JavaScript.**

Rta// 1. String: Es el que representa cadenas de texto, las cuales se pueden escribir con comillas simples ‘’ o dobles “”.

2. Number: Es el que representa valores numéricos ya sean enteros o decimales.

3. BigInt: Se usa para números muy grandes que excedan el límite de Number.

4. Boolean: Valores lógicos true o false.

5. Undefined: Valor que se le asigna automáticamente a una variable declarada pero no inicializada.

6. Null: Representa la ausencia intencional del valor.

7. Symbol: Valores únicos e inmutables que se usan como identificadores.

4. Ausencia de Valor: null vs. undefined **Aunque a menudo se confunden por los desarrolladores junior, null y undefined representan conceptos arquitectónicos distintos en JavaScript. Describa la diferencia teórica y de asignación entre ambos.**

Rta// Null es un valor intencionalmente asignado por el programador para indicar la ausencia deliberada de un objeto o valor.

Ej:

Let y = null;

Undefined es un valor primitivo asignado automáticamente en ciertas situaciones como variables no inicializadas o retorno de función faltante. Representa ausencia de asignación.

Ej:

Let x; //está declarada pero no inicializada.

Console.log(x);

5. Coerción de Tipos (Type Coercion) **Defina qué es la "coerción de tipos" implícita. Brinde una explicación teórica de cómo el motor de JavaScript resuelve internamente una operación matemática (como una suma o una resta) cuando uno de los operandos es un String y el otro es un Number.**

Rta// La coerción de tipos es cuando se convierte automáticamente un tipo de dato a otro, como de String a Number. Este permite trabajar con diferentes tipos de datos sin necesidad de poner conversiones explícitas.

El motor sigue el algoritmo ToPrimitive y ToString/ToNumber, es decir, si el operador es suma (+) y hay un String, se aplica ToString al otro operando, pero si el operador es aritmético (-, *, /), se aplica ToNumber a ambos operandos.

6. El operador typeof y su anomalía histórica **¿Cuál es el propósito principal del operador typeof en tiempo de ejecución? Adicionalmente, mencione cuál es el resultado al evaluar typeof null y por qué este comportamiento es considerado un error fundacional (bug histórico) en el diseño del lenguaje.**

Rta// El operador typeof tiene como propósito principal identificar el tipo de dato de una expresión en tiempo de ejecución, devolviendo una cadena de texto indicando el tipo de valor que se le pasa.

El resultado al evaluar typeof null es “object”, este error se remonta a los años 90, en donde el lenguaje de programación fue diseñado muy rápido, y los detalles internos dejaron rarezas.

Internamente, los valores de JavaScript se representaban mediante una type tag en forma binaria, en donde la etiqueta para objetos era 000 y null también se representaba con un valor binario de ceros 0x00. Como resultado, hizo que null se interpretara como un objeto, así que es un error en la implementación original.

7. Elevación (Hoisting) y la Zona Muerta Temporal (TDZ) **Explique el concepto de "Hoisting" durante la fase de compilación del motor de JavaScript. ¿Cómo afecta este comportamiento de manera diferente a las variables declaradas con var frente a las declaradas con let o const (haciendo referencia a la Zona Muerta Temporal)?**

Rta// Hoisting (elevación) se refiere al comportamiento del motor durante la fase de compilación, en la cual las declaraciones de variables y funciones se mueven o se registran al inicio del contexto de ejecución antes de que el código se ejecute. Esto significa que se pueden usar ciertas variables o funciones antes de que aparezcan en el código fuente.

La Zona Muerta Temporal es el período entre la entrada de un scope y la declaración de una variable usando let o const. Durante este periodo la variable se encuentra en un estado no inicializado y cualquier intento de acceder a ella resultará en un error de referencia.

Hoisting para var: La variable declarada con var es elevada al inicio de su scope funcional o global y son inicializadas automáticamente con valor undefined. Este comportamiento puede producir errores lógicos difíciles de detectar, ya que la variable existe, pero aún no ha recibido el valor asignado explícitamente.

Hoisting con let: El código registra la variable antes de declararse con let, causando un error de referencia. Esto pasa porque las variables que son elevadas, pero no inicializadas permanecen en la TDZ hasta que se ejecute la declaración.

Hoisting con const: La variable x se declara con const, que es con block scope y no elevada. Cuando se llama a la variable antes de la declaración se genera un error de referencia como con en let, ya que se encuentra en la zona muestra temporal (TDZ).

8. Reglas de Identificadores (Naming Conventions) **En cuanto a la declaración de identificadores (nombres de variables), mencione tres (3) reglas estrictas que provocarían un error de sintaxis inmediato si se incumplen. Asimismo, indique cuál es el estándar de capitalización adoptado por la industria para nombrar variables comunes en JavaScript.**

Rta// **1.** No pueden comenzar con un número.

Let 1edad = 20;

2. No pueden contener espacios ni caracteres especiales (solo $ y _).

Let nombre# = "inválido";

3. No pueden ser palabras reservadas del lenguaje (let, class, return, function).

Let return = 095;

El estándar de capitalización es el **camelCase,** es decir, que la primera palabra comienza en minúscula y cada palabra siguiente inicia con mayúscula, sin espacios ni guiones.

nombreUsuario

precioTotal

9. La "Inmutabilidad" relativa de const **Teóricamente, si declaramos una variable usando const, no podemos reasignarle un nuevo valor. Sin embargo, ¿qué sucede en la memoria si usamos const para declarar una estructura de datos compleja (como un Objeto o un Arreglo) y luego intentamos modificar sus propiedades o elementos internos? Justifique su respuesta.**

Rta// El const protege la referencia, no el contenido, lo que sucede en la memoria es que al declarar un arreglo lo que queda "congelado" es la dirección

de memoria, la cual no se puede cambiar por otra. Sin embargo, el contenido que vive en esa dirección dentro del heap sigue siendo maleable.

No se puede hacer reasignación, hacer que una variable apunte a un lugar diferente, la memoria lanzará un TypeError.

Sin embargo, si se puedo hacer mutación, es decir, modificar las propiedades internas no cambia la referencia, solo se modifican los datos almacenados en la misma ubicación de memoria, así que el lenguaje lo permite sin generar error.

10. La paradoja de NaN **El valor especial NaN es un acrónimo de "Not-a-Number". Teóricamente, ¿bajo qué circunstancias el motor de JavaScript arroja este valor? Además, indique qué tipo de dato retorna si le aplicamos el operador typeof a un valor NaN**

Rta// Este valor se arroja por varias circunstancias principales que son: operaciones matemáticas imposibles, como la raíz cuadrada de un número negativo o el logaritmo de un número negativo; operaciones aritméticas con valores no numéricos como “cinco” * 2; operaciones indeterminadas como 0/0; y cuando se intenta forzar un valor a número y este no tiene una representación numérica lógica.

Por otro lado, si se le aplica el typeof a un valor NaN, el resultado será “Number”, ya que es un estado específico dentro del tipo de dato Number, por lo que este es un valor numérico especial.

Conclusiones

Para concluir este taller sobre los fundamentos y arquitectura de datos en JavaScript, podemos determinar que la evolución del lenguaje refleja una búsqueda constante de mayor predictibilidad y seguridad en el desarrollo de software, especialmente en la utilización de var, let y const para obtener un mejor control del alcance de bloque y evitar errores lógicos derivados del Hoisting. JavaScript siendo de tipado dinámico ofrece una flexibilidad a comparación de lenguajes de tipado estático como Java, permitiendo reasignaciones en tiempo de ejecución, aunque esto también permite que nosotros como desarrolladores podamos comprender términos importantes para la utilización de este lenguaje de programación.

Finalmente, vemos como el estudio de estructura de datos complejas y las anomalías históricas nos muestra la importancia de dominar los aspectos técnicos y el funcionamiento del motor de JavaScript para construir aplicaciones robustas, escalables y alineadas con los estándares actuales, asimismo, haciendo que el código sea lo más completo y estable posible, en el cual se haga un buen uso de la memoria.

Bibliografía

Codechappie. (s.f.). *¿Cuál es la diferencia entre let, var y const en JavaScript?*. Codechappie Blog. https://www.codechappie.com/blog/cual-es-la-diferencia-entre-let-var-y-const-en-javascript

CódigoDev. (s.f.). *Variables y operadores en JavaScript: ejercicios resueltos*. CódigoDev. https://codigodev.com/variables-y-operadores-en-javascript-ejercicios-resueltos/#Cual_es_la_diferencia_entre_var_let_y_const

GeeksforGeeks. (s.f.). *Difference between var, let and const keywords in JavaScript*. GeeksforGeeks. https://www.geeksforgeeks.org/javascript/difference-between-var-let-and-const-keywords-in-javascript/

GeeksforGeeks. (s.f.). *Temporal Dead Zone in JavaScript*. GeeksforGeeks. https://www.geeksforgeeks.org/javascript/temporal-dead-zone-in-javascript/

GeeksforGeeks. (s.f.). *Undefined vs Null in JavaScript*. GeeksforGeeks. https://www.geeksforgeeks.org/javascript/undefined-vs-null-in-javascript/

KeepCoding. (s.f.). *NaN en JavaScript: cómo evitarlo y gestionarlo*. KeepCoding Blog. https://keepcoding.io/blog/nan-en-javascript-como-evitarlo-y-gestionarlo/

Leolas95. (s.f.). *Tipado estático y dinámico, tipado fuerte y débil*. DEV Community. https://dev.to/leolas95/tipado-estatico-y-dinamico-tipado-fuerte-y-debil-133n

MDN Web Docs. (s.f.). *const*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const

MDN Web Docs. (s.f.). *JavaScript data types and data structures*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures

Moslah, M. (s.f.). *Why typeof null === 'object' in JavaScript? A deep dive into a quirky bug*. DEV Community. https://dev.to/m0slah/why-typeof-null-object-in-javascript-a-deep-dive-into-a-quirky-bug-n6p

Plata, R. (s.f.). *¿Por qué deberíamos evitar el uso de var en JavaScript?*. LinkedIn. https://www.linkedin.com/pulse/por-qu%C3%A9-deber%C3%ADamos-evitar-el-uso-de-var-en-javascript-rey-plata-dlubf

Platzi. (s.f.). *Deja de usar var en JavaScript: por qué no es buena práctica usarlo*. Platzi Blog. https://platzi.com/blog/deja-de-usar-var-en-javascript-por-que-no-es-buena-practica-usarlo/

W3Docs. (s.f.). *Conversión de objeto a primitivo*. W3Docs. https://es.w3docs.com/aprende-javascript/object-to-primitive-conversion.html